EDP-Client File Upload To S3 Bucket

Table Of Contents

- Pattern Summary
  - Description
    - Three Bucket System
  - Diagram
  - Use Cases
  - Goals & Benefits
  - Limitations
  - Components/Services
  - When To Use
  - Related Patterns
- Security
- Cost Model
- Deployment Strategy
- Review History
- Appendix
  - References

Column Name	*Column Value*	*Column Reference Guide*
Pattern ID	EDP-5	
Pattern Name	Client File Upload To S3 Bucket	
Capability(Primary)	Security, Identity, Access, Storage	
Capability(Secondary)	Transient Storage	
Cloud Specific	Yes	
Cloud Service Provider(s)	AWS, GPC	
Service Offering(s)	S3, Apigee	
Status	APPROVED	
Solution Architect	@Thomas Zablocki	
Reviewer(s)	@Paul Labossiere , @Jeffery Rack , @Dale Hoshooley , @Bhagi Raj Limbu , @Amarendra Sabat , @Jason Weed Ling	
Approver(s)	@Wallace Wong , @Jeffery Rack , @Paul Labossiere	
Approved Date	Apr 9, 2024	

Pattern Summary

Description

NOTE: This is a strategic pattern to allow transient storage for any system. This pattern is only for the process of storing a file in S3 and not what happens to it afterward, those will be separate patterns (such as 'S3 to AV scan', S3 to FileNet' etc)

Canada Life life today is closely coupled with a system that stores files (FileNet, OnBase, Salesforce).

The solution where document files are stored in Salesforce directly (Advisor Workspace, MCLAW Contact Us) is not meeting Vulnerability Scan criteria (no checks for virus and no check for file type).

Due to close coupling with FileNet MCLAW file upload for claim submission, claim estimate and audit response by user are immediately not available to the user when FileNet service is not available. This is not ideal, especially with a strict SLA for Feds user base.

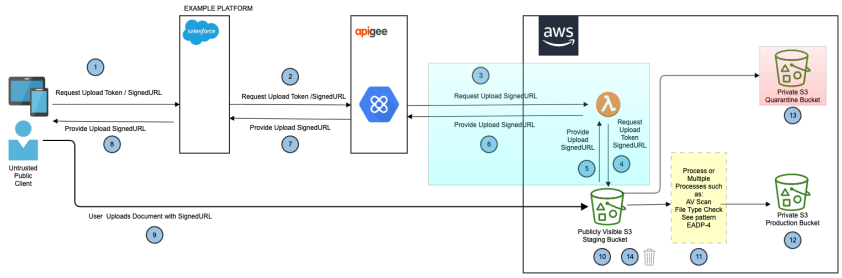
Three Bucket System

The three bucket system allows a physical separation between the ingestion storage, quarantined storage and production usage storage.

The pattern allows separating incoming objects to publicly discoverable and write only bucket, while the Production Bucket would be private and not discoverable and read only by the consumers.

Another pattern such as AV Scan or File Type check could occur prior to storing the uploaded file in Production Bucket.

Diagram



Step	Description	Notes
1	A 3rd party raises a file upload request to it's backend. This could be any platform (React or Angular Components, Salesforce LWC).	
2	The backend on behalf of the client request a token or signed url for upload functionality via a Proxy. This will be a short life authorization. The request would go to Apigee as with other similar actions today in Production.	
3	The proxy would trigger a preconfigured serverless Lambda function to request an upload signedURL from AWS S3 bucket. • The lambda function would have a predefined role to be allowed to request a signedURL, a client would not have the right to request such a permission directly.	Lambda could be replaced here by having Apigee directly request SignedURL from S3 bucket on the client's behalf. This is to be confirmed by API team if possible.
4	Lambda function requests an upload signedURL from AWS S3 bucket.	
5	Staging S3 Bucket provides Lambda function with the upload signedURL.	
6	Lambda function to returns the upload signedURL from AWS S3 bucket to Apigee Proxy / Gateway.	
7	Apigee Proxy / Gateway returns the upload signedURL to the client's backend.	
8	The client's backend provides the client with a short life signedURL that allows the client to upload file(s) to the publicly visible S3 Staging Bucket.	
9	The client can save a document directly to Public S3 Staging Bucket with provided signedURL.	
10	Unscanned and unprocessed file is saved at Public S3 Staging Bucket.	
11	This could represent a number of processes performed on the file such as EADP-4 Anti Virus Scan or File Type Check, and more.	
12	After a file passed all the necessary checks and/or finished processing it is stored in Private S3 Production Bucket.	
13	Any file that is deemed not usable (didn't pass virus check, or failed other checks) is stored in Private S3 Quarantine Bucket.	
14	The file is now deleted from the Public S3 Staging Bucket.	

Note: This pattern is only for the upload portion, not what happens after it is stored.

Use Cases

MCLAW:

- Contact Us File Upload (also used via Contact Us → Appeals (Feds requirement))
- Audit File Upload
- Claim Submission Upload
- Claim Estimate Upload
- Benefit Coverage Reduction (upcoming)

Flywheel:

- Covers different channels call centre, fax, email, mail, GRSA + pattern would be reused for Disability

Advisor Workspace:

- File Upload of scanned and file typed documents.

Goals & Benefits

Secure way to store files prior usage.

Provides a pattern for transient storage in AWS.

Provides entry point for other patterns such as Virus Scanning, File Type checking or any other process needed to be performed on a file prior its usage.

Limitations

This is an upload document pattern only not an E2E pattern.

Components/Services

Apigee Gateway and APIs

AWS Lambda (optional)

AWS S3 Bucket

When To Use

Transient storage is need to abstract or decouple destination of the document to the user.

Document destination storage is volatile or SLA is low.

Large documents are limited by Salesforce Heap Size 4.3mb today.

Cloud storage is needed.

Related Patterns

EADP-4.
Document Upload To FileNet

Security

SignedURL, Token via Apigee Access Token

Cost Model

- Chargeback (if any) to business unit for deploying this pattern

Deployment Strategy

- Overview of how this pattern can be brought to life including link to any reference implementation e.g. Terraform script to spin up environment for this pattern

Review History

Version #	Changes	Solution Architect	Review Date	Reviewer(s)	Approved Date	Approver(s)	Notes

Appendix

References

- Links to documentation leveraged to develop this pattern